Lecture 6: Shortest Paths & DBScan

Dijkstra's Algorithm

- Algorithm: **Breadth First Search**
  - For Special Case: $w(e) = 1$ for all $e \in E$
- Algorithm: **Dijkstra**
  - general case $w(e) \geq 0$ for all $e \in E$

```
Dijkstra(WeightedGraph G = (V, E; w), Vertex s)
  Initialize(G, s)
  Q = new PriorityQueue(V, d)

  while Q ≠ ∅ do
      u = Q.ExtractMin()
      foreach v ∈ u.neighbors do
          if v.d > u.d + w(u, v) then
              v.color = gray
              v.d = u.d + w(u, v)
              v.π = u
              Q.DecreaseKey(v, v.d)
      u.color = black
```

Path s-u-v is better than known s-v-path!

v moves forward in priority queue.

Proof of Correctness

Let $dist(s, u)$ be the distance from s to u via a shortest path.

Simplification: We assume that no 2 nodes have same distance from s: $dist(s, v_1) < dist(s, v_2) < \ldots < dist(s, v_n)$, where v_i is the i-th nearest node from s.

Loop Invariant

In iteration i of the while-loop:

- A) the attribute $v_i.d$ is equal to $dist(s, v_i)$
- B) v_i is extracted from Q and its color is set to black

Proof by Induction

Inductive step:

We observe that, in a shortest $s - v_i$-path, the predecessor u of v_i is in $S := \{v_i, v_2, ..., v_{i-1}\}$, i.e. , among the $i - 1$ nearest nodes from s.

Due to the loop invariant, $u.d = dist(s, u)$ held when the edge (u,v_i) was relaxed, resulting in $v_i.d = u.d + w(u, v_i) = dist(s, v_i) \Rightarrow A$

For any node $w \notin S$ other than v_i holds at iteration i that $w.d \geq dist(s, w) > dist(s, v_i) = v_i.d \Rightarrow B$

Running Time

```
Dijkstra(WeightedGraph G = (V, E; w), Vertex s)
    Initialize(G, s)                                   ] O(|V|)
    Q = new PriorityQueue(V, d)

    while Q ≠ ∅ do
        u = Q.ExtractMin()                             ] O(|V| · T_ExtractMin(|V|))
        foreach v ∈ u.neighbors do
            if v.d > u.d + w(u, v) then
                v.color = gray                         ] O(|E|)
                v.d = u.d + w(u, v)
                v.π = u
                Q.DecreaseKey(v, v.d)                  ] O(|E| · T_DecreaseKey(|V|))
        u.color = black
```

$$\Sigma = O(|V| \cdot T_{\text{ExtractMin}}(|V|) + |E| \cdot T_{\text{DecreaseKey}}(|V|))$$

$$T_{Dijkstra}(|V|, |E|) = \mathcal{O}(|V| \cdot T_{ExtractMin}(|V|) + |E| \cdot T_{DecreaseKey}(|V|))$$

$$T_{\text{Dijkstra}}(|V|,|E|) = O(|V| \cdot T_{\text{ExtractMin}}(|V|) + |E| \cdot T_{\text{DecreaseKey}}(|V|))$$

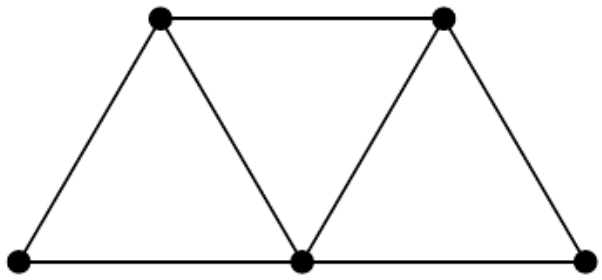
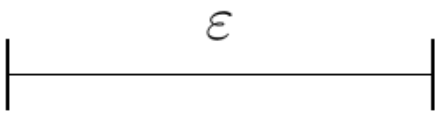
Priority queue implemented as ...	$T_{\text{ExtractMin}}(n)$	$T_{\text{DecreaseKey}}(n)$	$T_{\text{Dijkstra}}(\lvert V\rvert,\lvert E\rvert)$
... unsorted array	$O(n)$	$O(1)$	$O(\lvert V\rvert^2)$
... heap	$O(\log n)$	$O(\log n)$	$O(\lvert V\rvert \log \lvert V\rvert)$
... Fibonacci heap	$O(\log n)$ *amortised*	$O(1)$ *amortised*	$O(\lvert V\rvert \log \lvert V\rvert)$ *Worst Case!*

for planar graphs: $|E| \leq 3|V| - 6 \in O(|V|)$

DBScan

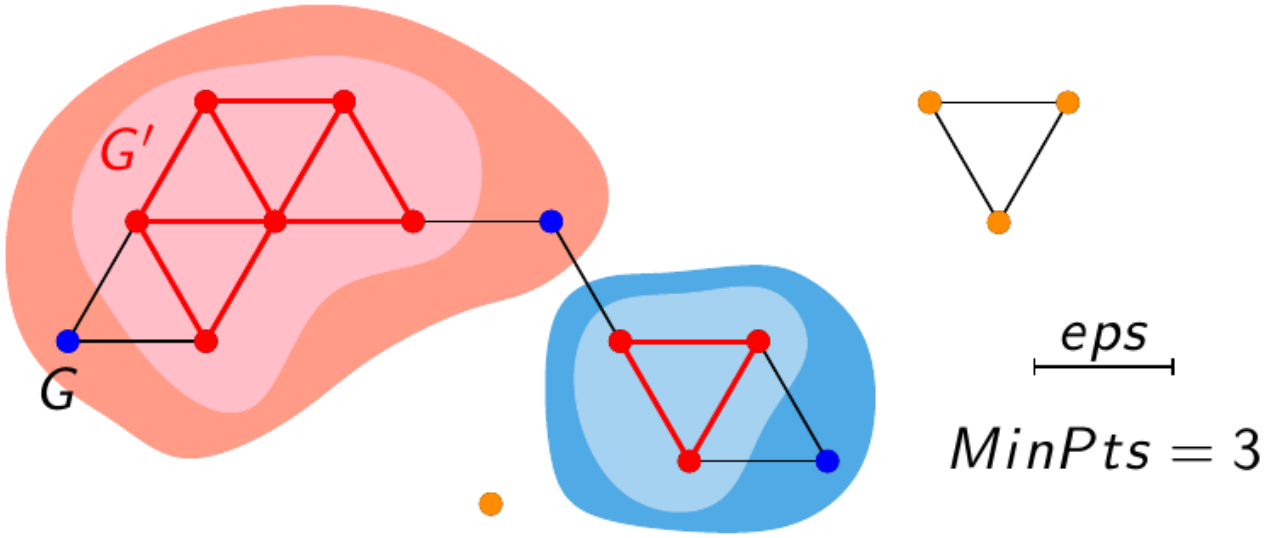
- Input: 2D reduced point cloud from Airborne LiDAR after removal of ground points
- Aim: Partition point set into clusters (Point Clustering)

The ϵ- neighborhood graph

- Definitions:
  - P is a set of points
  - ϵ-neighbourhood graph $G = (P, E)$ contains for two points $p, q \in P$ the edge $\{p, q\} \in E$ if $||p - q|| \leq \epsilon$
  - 


- Idea: Data Structure D(an index) that supports queries for all point in the ϵ-neighborhood of a point $p \in P$
  - Total running time: $\mathcal{O}(T_{set-up}(n) + nT_{query}(n))$, where:
    - $T_{set-up}(n)$ is the time needed to set up the data structure for n points and
    - $T_{query}(n)$: querying time depending on the number n of points in the data structure
  - Many application use D: Avoid computing neighborhood graph- find neighbours of point on demand (using a query)

DBScan

- Is density-based clustering method
- Design: Implementable with typical database queries
- DB = *density based* or *database*
- **Input**: point set P; parameters eps $\in \mathbb{R}_{>0}, MinPts \in \mathbb{N}$
- Definitions:
  - Neighborhood graph $G = (P, E)$ contains for points $p, q \in P$ edge $\{p, q\} \in E$ if $||p - q|| \leq eps$.
  - Points with at least $MinPts$ neightbors are **core points** (red points)
  - G' is subgraph of G induced by the sets of core points
  - Connected components of G' define clusters.
  - Non-core points with at least one core point as a neighbor are **reachable point** (blue)
    - Assigned to arbitrary neighbor cluster
  - All other points are **outliers**(yellow)
  - 

DBScan

Algorithm: Generally, the connected components of a graph can be found with a depth-first search.

```
DepthFirstSearch(Graph G = (V, E))
  foreach u ∈ V do
    u.visited = false
  foreach u ∈ V do
    DepthFirstSearchVisit(u, u.id)

DepthFirstSearchVisit(Node u, int id)
  if u.visited == false then
    u.visited = true
    u.center = id
    foreach v ∈ u.neighbors do
      DepthFirstSearchVisit(v, id)
```

For DBScan, we first need to find the adjacencies among the points, using spatial queries.